LAPORAN PRAKTIKUM

Modul 6

Disusun Untuk Memenuhi Tugas Kuliah

Mata Kuliah: Teknologi IoT

Dosen Pembimbing:

Agus Ambarwari, S.Pd., M.Kom.

Dr. Ir. Septafiansyah Dwi Putra



Oleh:

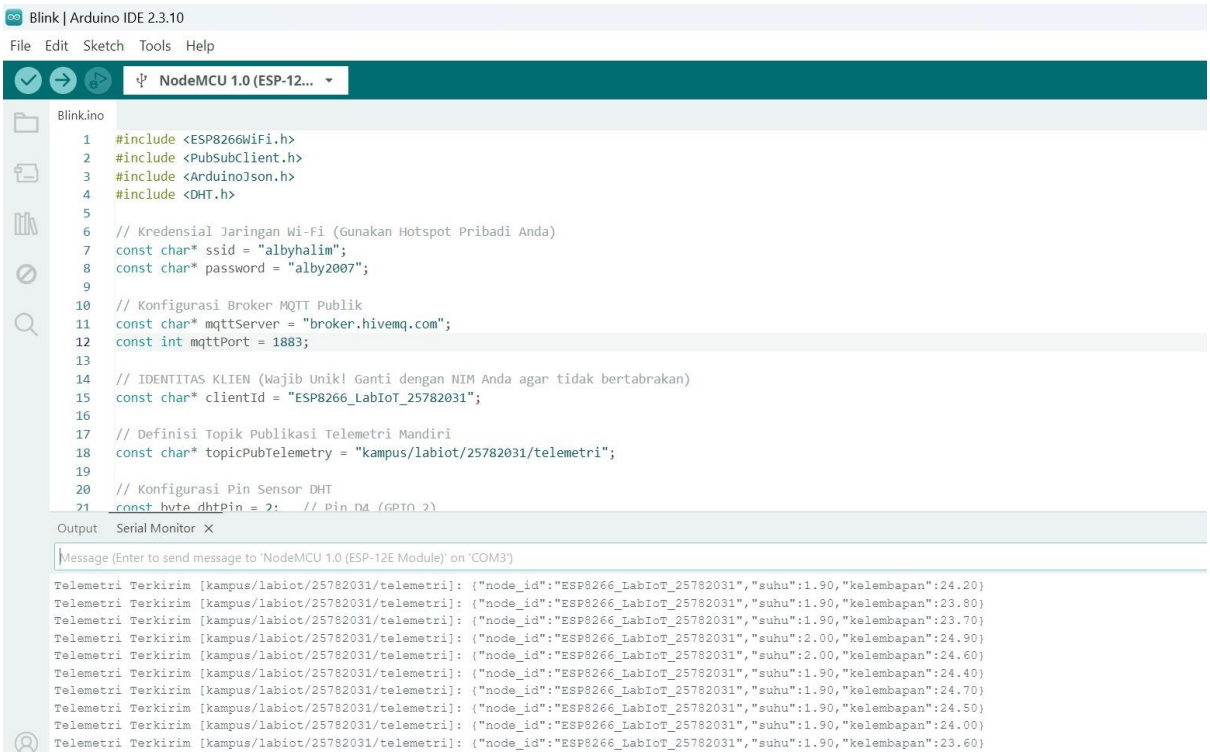
Alby Halim (25782031)

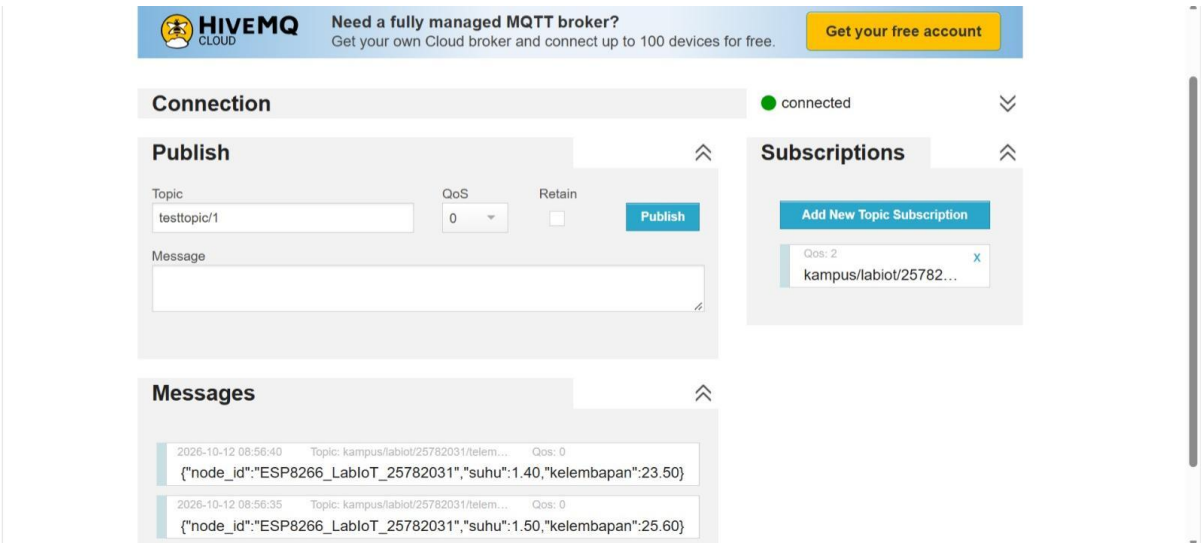
PROGRAM STUDI D4 TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGRI LAMPUNG

6.5.1 Praktikum 1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON





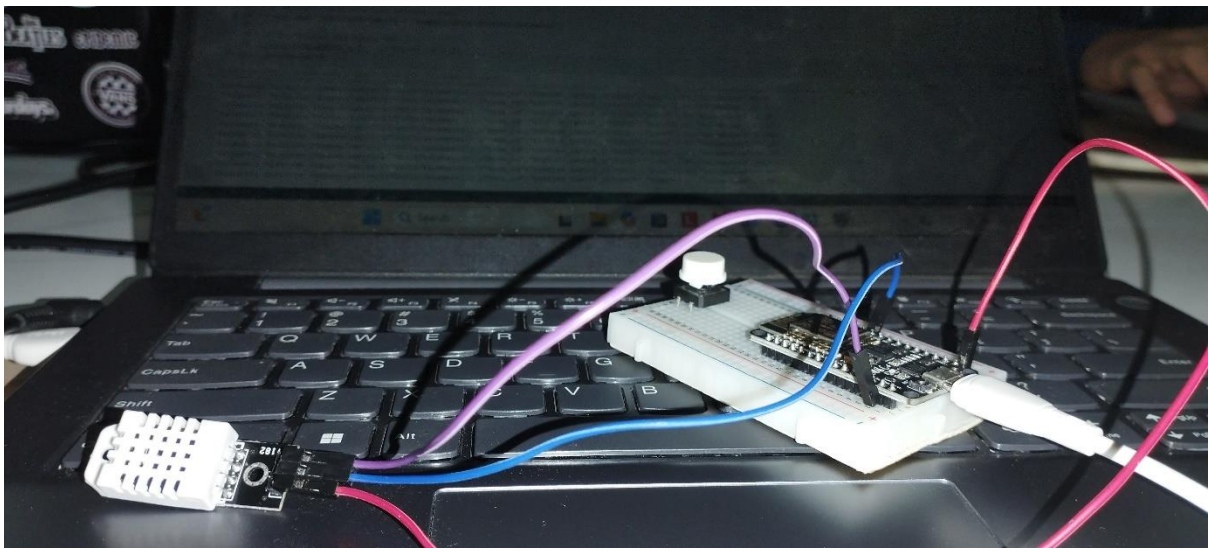
Disini user dapat memantau dan monitoring Sensor di dht melalui arduiono maupun dari broker, caranya masukan topic di subcriptions lalu tekan “publish” nantinya imessages akan menampilkan output hasil monitoring perangkat

Penjelasan Singkat Kode:

- WiFiClient espClient; : Membuka antarmuka soket jaringan TCP level dasar pada cip ESP8266.
- PubSubClient client(espClient); : Menginisialisasi objek protokol MQTT yang beroperasi di atas soket jaringan espClient.

- Client.setServer(mqttServer, mqttPort); : Menetapkan alamat host broker dan port layanan (
- 1883
- Merupakan port standar MQTT teks terbuka / tanpa TLS).
- Client.loop(); : Fungsi wajib. Rutinitas ini bertugas memproses paket TCP masuk, menjaga interval keep-alive ping dengan server agar koneksi tidak diputus paksa (timeout), serta mengelola antrean pesan.
- StaticJsonDocument<256> doc; : Mengalokasikan blok memori tetap di SRAM (sebesar
- 256Byte) untuk menampung pohon objek JSON tanpa memicu fragmentasi memori heap.
- serializeJson(doc, jsonBuffer); : Mengubah pohon objek JSON menjadi representasi string karakter padat yang siap ditransmisikan.

Foto Rangkaian



6.5.2 Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron

Blink | Arduino IDE 2.3.10

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...

Blink.ino

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

// Kredensial Jaringan Wi-Fi
const char* ssid = "albyhalim";
const char* password = "alby2007";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782031_Sub";

// Definisi Topik Subskripsi Perintah Mandiri
const char* topicSubCommand = "kampus/labiot/25782031/perintah";

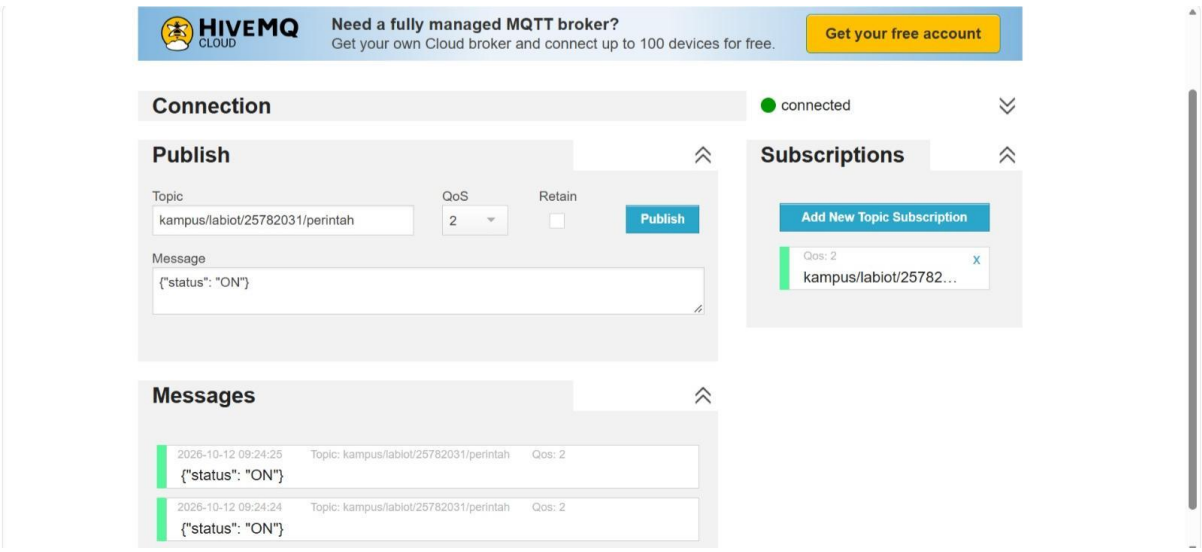
// Konfigurasi Pin Indikator LED
const byte ledPin = 5; // Pin D1 (GPIO 5)

WiFiClient espClient;
PubSubClient client(espClient);
```

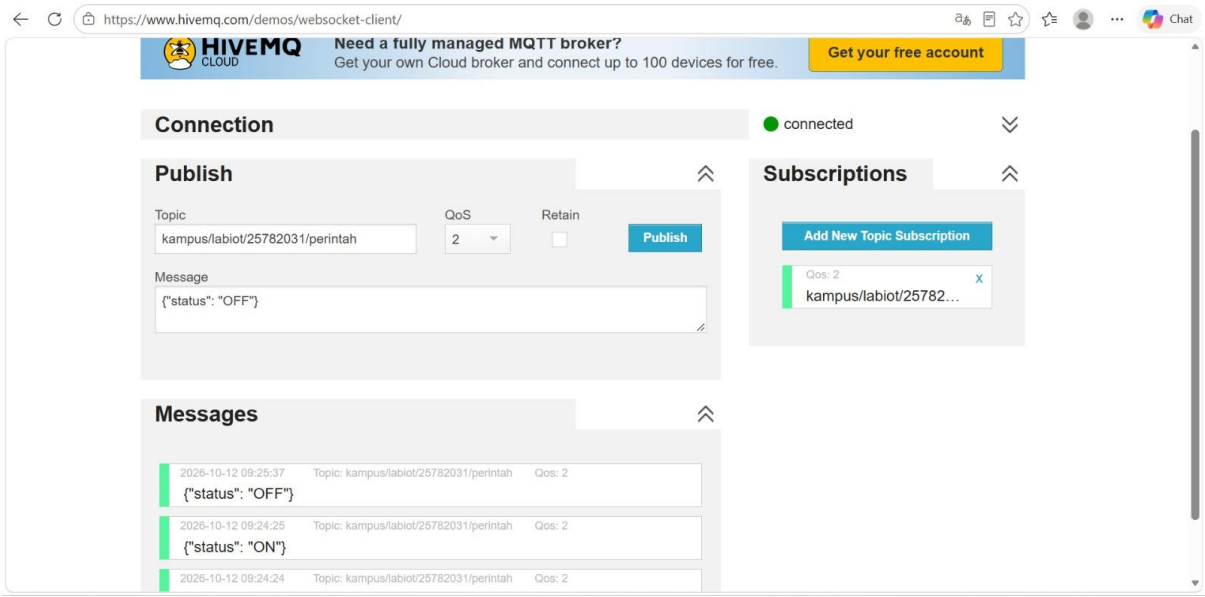
Output Serial Monitor

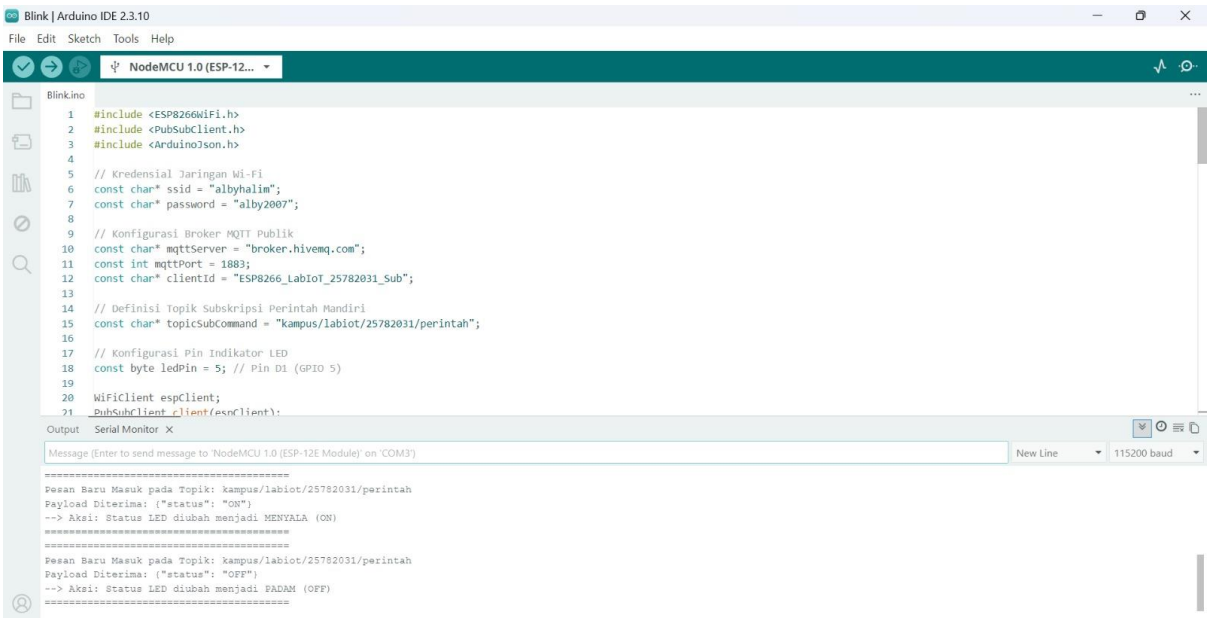
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

```
.....
Wi-Fi Terkoneksi!
Mencoba koneksi MQTT.........
Wi-Fi Terkoneksi!
Mencoba koneksi MQTT... Terhubung!
Subskripsi Topik: kampus/labiot/25782031/perintah
```



Pada praktikum ini, NodeMCU ESP8266 diprogram sebagai Subscriber. Mikrokontroler mendaftarkan minatnya pada topik perintah (kampus/labiot/nim_anda/perintah). Saat pengguna luar (melalui aplikasi penguji MQTT) memublikasikan perintah berformat JSON, fungsi callback NodeMCU akan membedah payload dan menyalakan/mematikan lampu LED indikator secara seketika. Disini masih pada Hivemq user dapat mengisi message dengan format JSON untuk menghidupkan atau memadamkan lampu LED dan pesan monitoringnya akan muncul pasa arduiono dan Hivemq





Penjelasan Singkat Kode:

- Client.setCallback(onMessageReceived); : Mendaftarkan fungsi penangan paket data masuk (callback function). Kapan pun broker mengirimkan pesan yang sesuai dengan topik subskripsi, fungsi onMessageReceived akan dipicu secara otomatis.
- Client.subscribe(topicSubCommand); : Mengirimkan instruksi subskripsi ke broker bahwa klien bersedia menerima semua pesan yang dialirkan ke topik tersebut.
- deserializeJson(doc, payloadStr); : Mengurai teks string JSON mentah dan menyusunnya kembali menjadi struktur pohon data yang kuncinya dapat diperiksa melalui doc.containsKey() dan nilainya dapat diekstrak secara langsung.

Foto Rangkaian (ON)

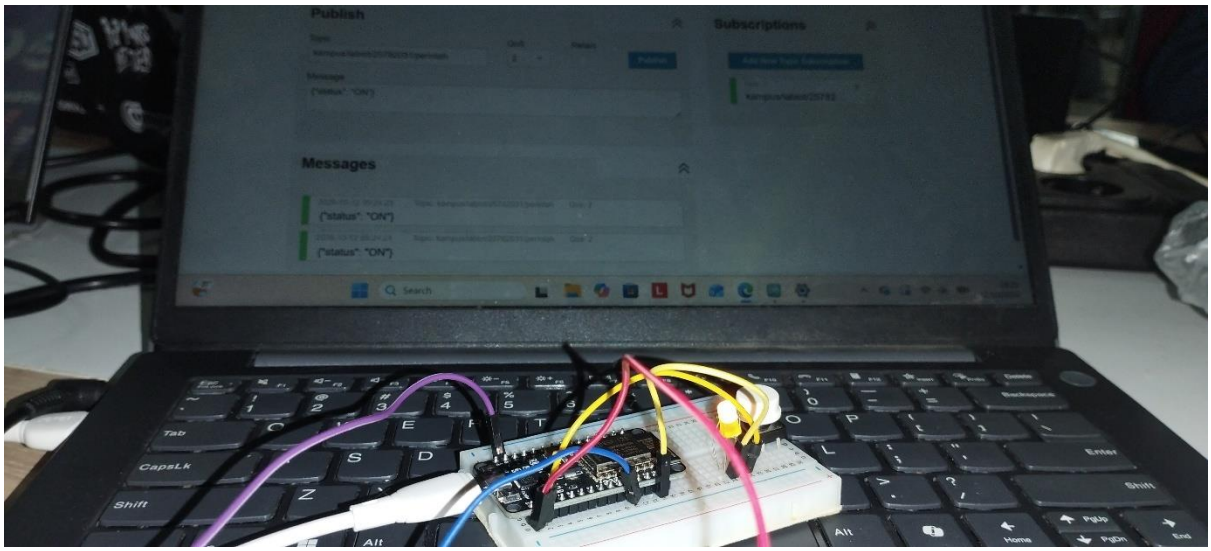
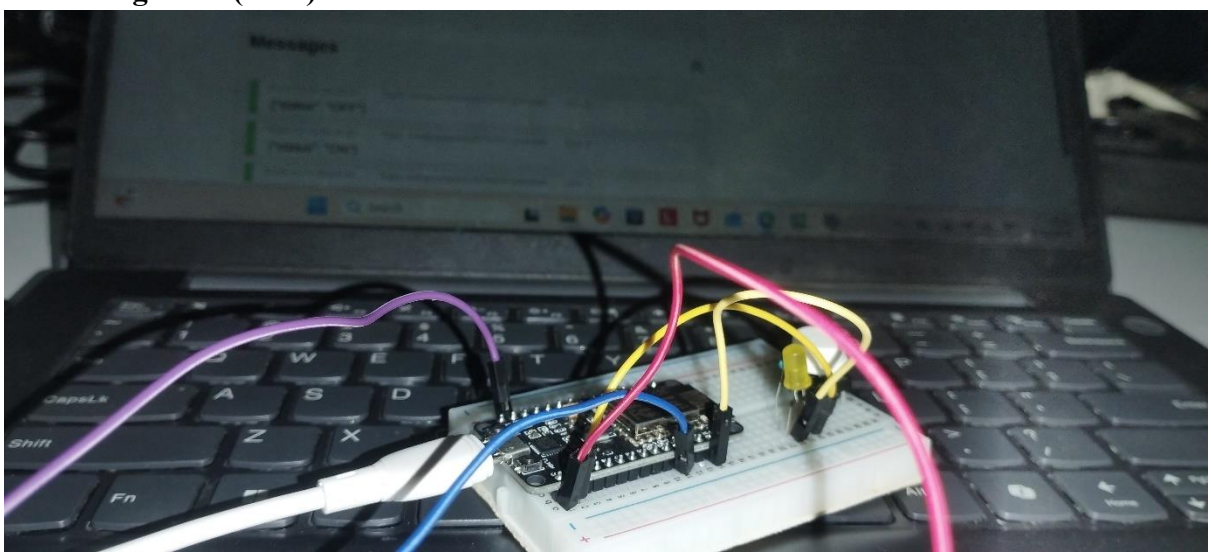
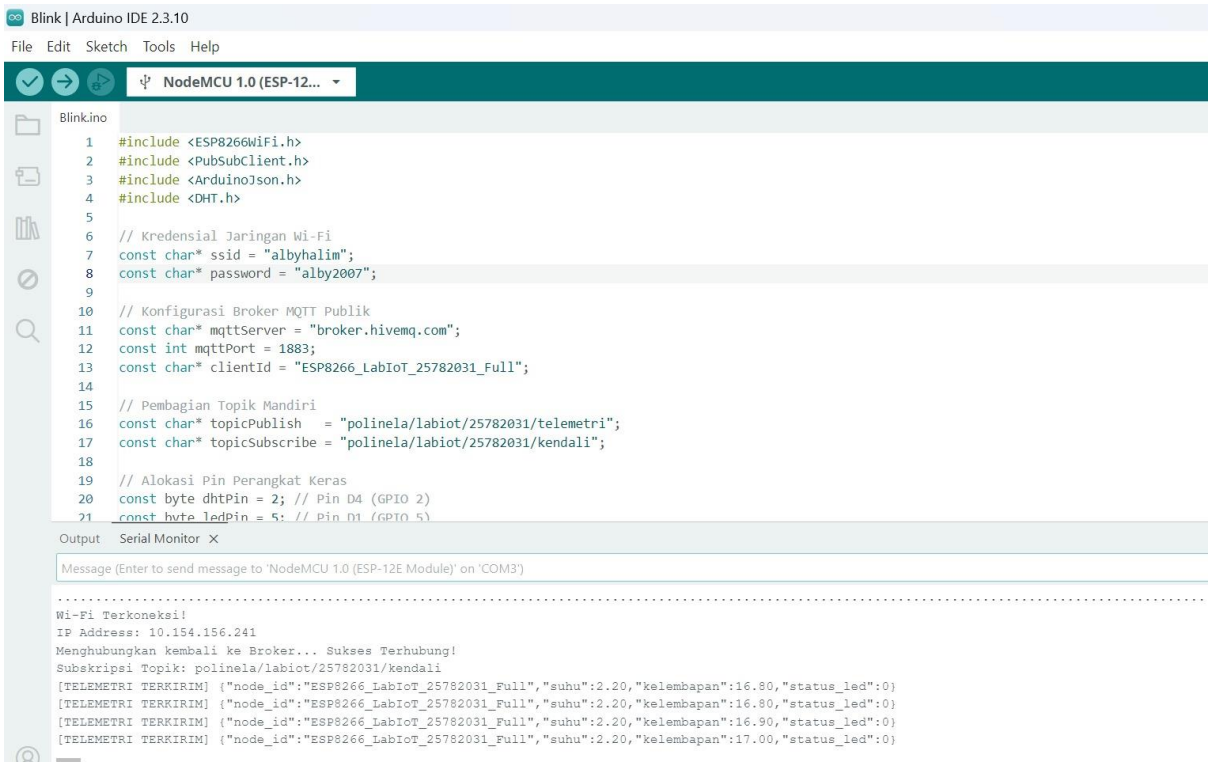
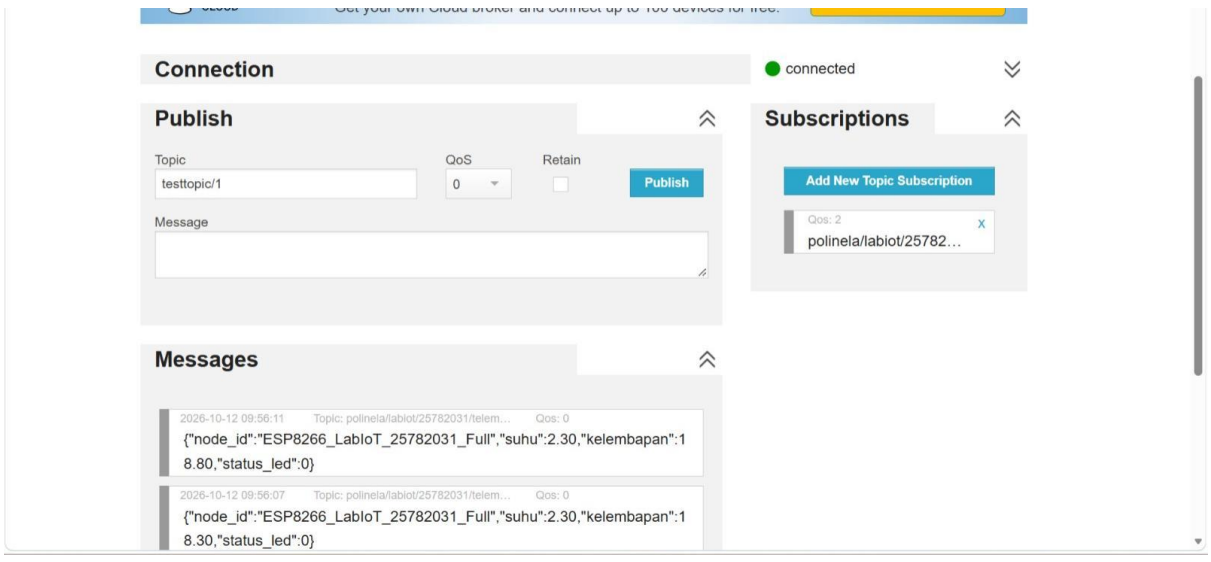


Foto Rangkaian (OFF)

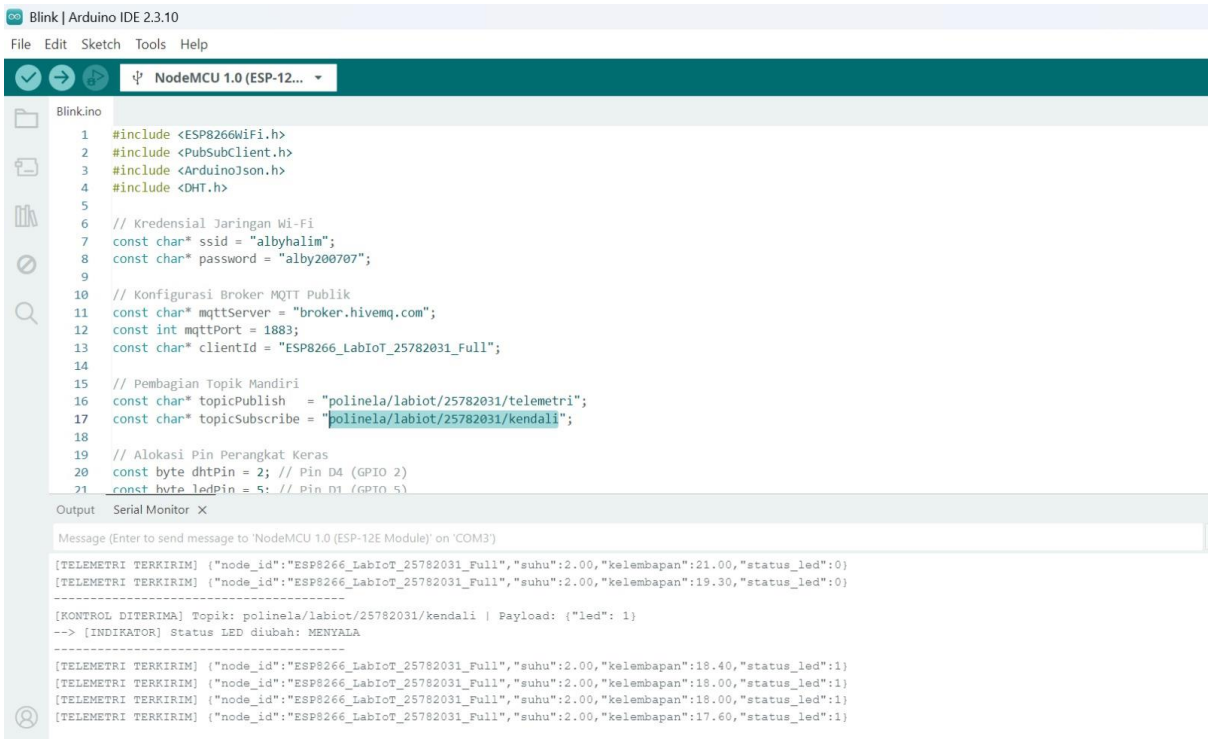


6.5.3 Praktikum 3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)

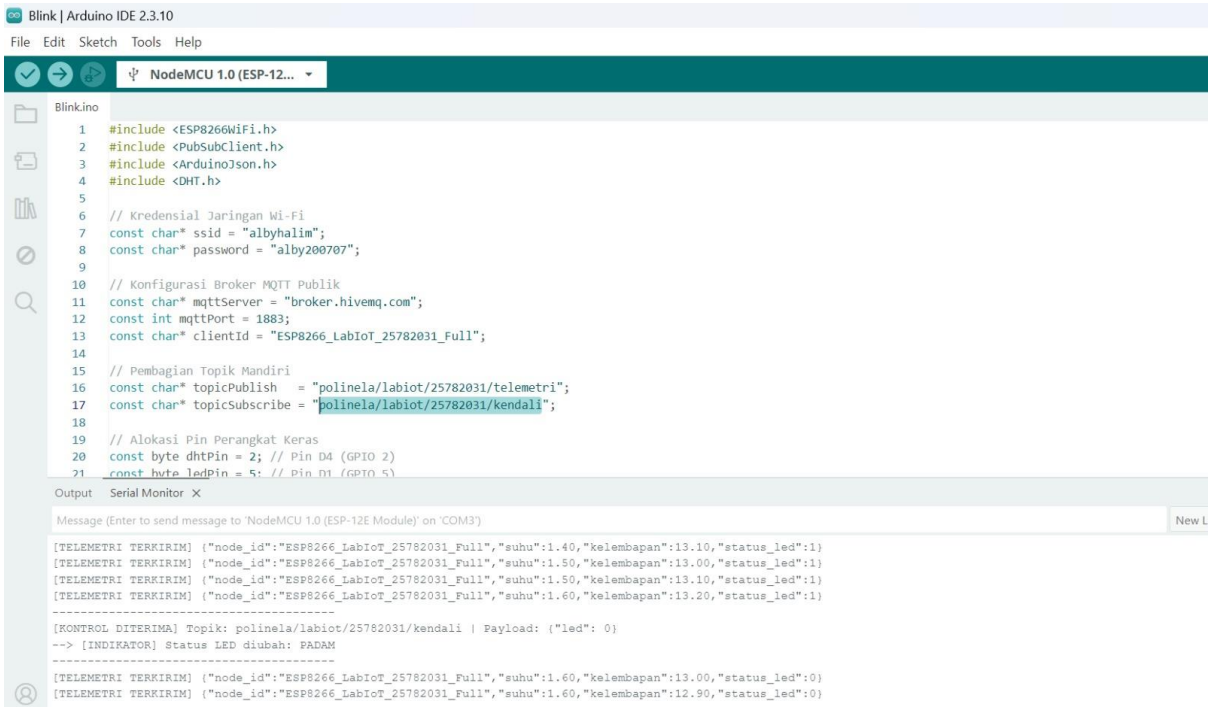


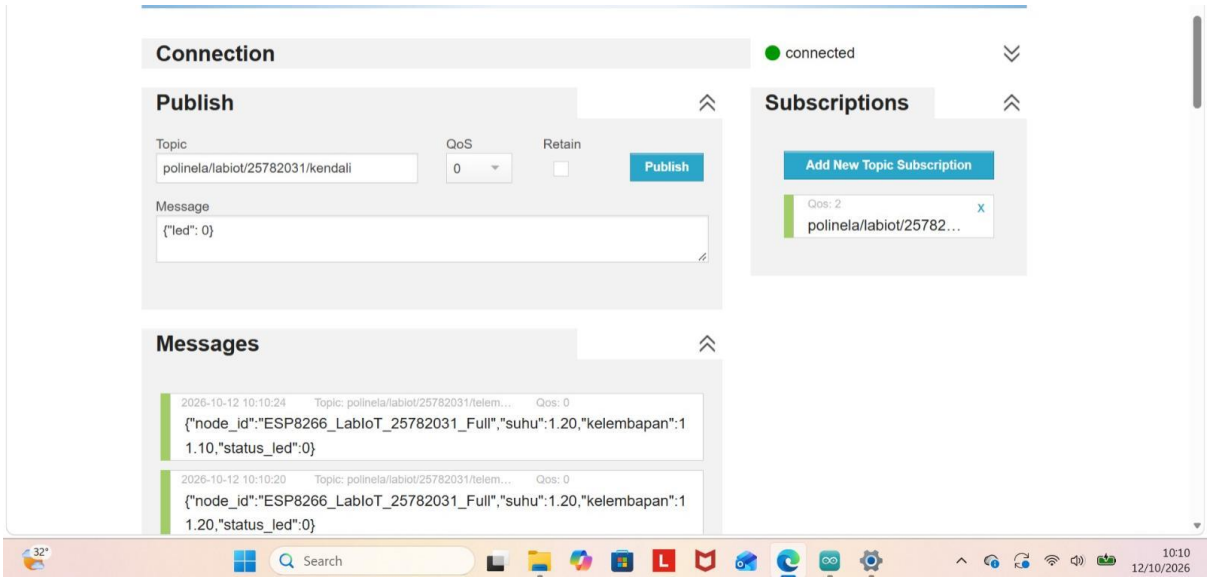


Led Menyala



Led Padam





Pada subscriberion user dapat mengisi Telemetry dan pada bagian topik kendali, lalu masih sama seperti praktikum sebelumnya pada bagian msessage user dapat mengisi format JSON untuk menghidupkan atau memadamkan led, 1 untuk LED ON dan 0 untuk LED OFF

Foto Rangkaian (LED ON)

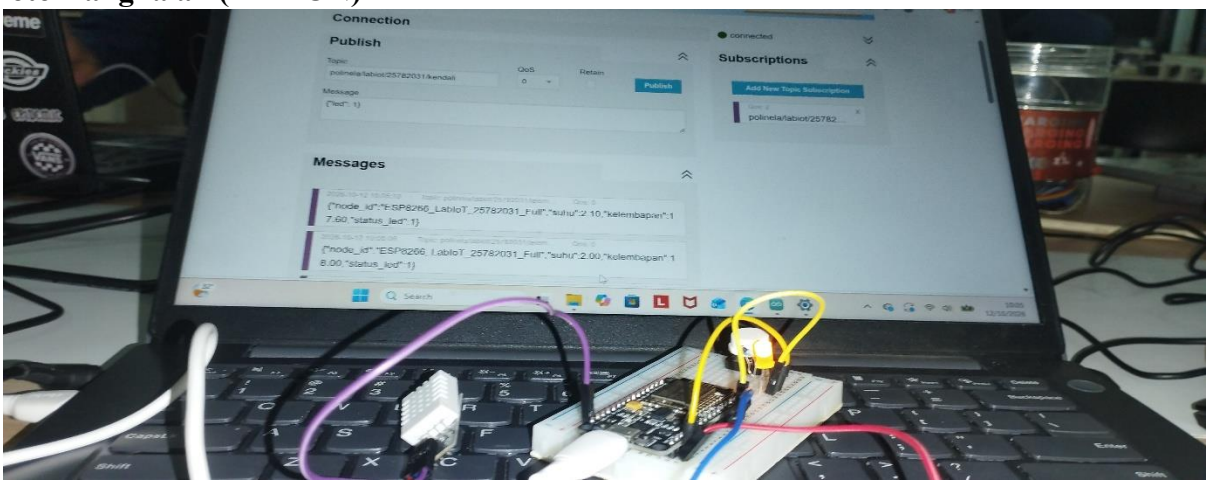
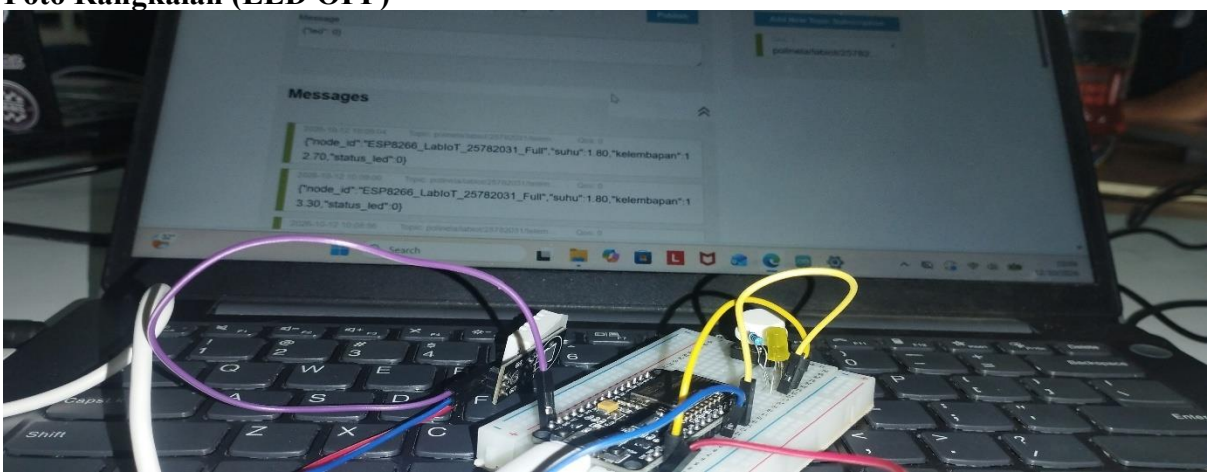


Foto Rangkaian (LED OFF)



6.6 Latihan

Latihan 1 (Analisis Arsitektur & Perbandingan Protokol):

Pada Modul 3, kita menggunakan protokol HTTP untuk membaca dan mengontrol perangkat. Jelaskan secara mendalam mengapa protokol MQTT jauh lebih unggul daripada HTTP dalam hal:

1. Efisiensi Penggunaan Kapasitas Bandwidth pada Paket Data Seluler:

Protokol HTTP berbasis Request-Response memiliki beban header (overhead) yang sangat besar (berkisar antara $200 - 800\ \text{byte}$ per paket transmisi karena memuat berbagai metadata teks seperti cookies, user-agent, dan status).

Sebaliknya, MQTT dirancang dengan ukuran header yang sangat minimal (serendah $2\ \text{byte}$), sehingga sangat efisien dalam menghemat konsumsi kuota data seluler, terutama pada perangkat IoT skala besar yang mengirimkan data secara berulang-ulang.

2. Beban Komputasi Mikrokontroler (CPU Overhead) saat Menangani Pertukaran Data Dua Arah:

Protokol HTTP bersifat stateless dan berbasis teks terbuka yang membutuhkan proses parsing string yang berat dan pembukaan/penutupan koneksi TCP berulang kali untuk setiap permintaan baru, sehingga membebani sumber daya CPU dan memori mikrokontroler yang terbatas.

3. Kemampuan Pengiriman Data Secara Simultan ke Banyak Perangkat Penerima (1-to-N distribution):

HTTP menggunakan komunikasi point-to-point (satu pengirim ke satu peladen/klien), sehingga menyulitkan distribusi data ke banyak penerima secara bersamaan tanpa menambah kompleksitas arsitektur peladen.

Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):

Sebuah pabrik manufaktur memiliki 3 lini produksi (line1, line2, line3), di mana setiap lini memiliki 4 mesin (m1, m2, m3, m4), dan setiap mesin melaporkan metrik suhu, getaran, dan status. Struktur topiknya adalah pabrik/<line>/<mesin>/<sensor>. Tentukan ekspresi topik subskripsi (subscription topic) yang tepat menggunakan karakter wildcard (+ atau #) untuk:

1. Memantau parameter suhu dari seluruh mesin di semua lini produksi:

Ekspresi topik subskripsi: pabrik/+/+/suhu Penjelasan: Karakter + pertama menggantikan semua variasi lini (line1, line2, line3), karakter + kedua menggantikan semua variasi mesin (m1, m2, m3, m4), dan diikuti dengan parameter spesifik suhu.

2. Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin m1 di lini line2:

Ekspresi topik subskripsi: pabrik/line2/m1/#

Penjelasan: Jalur spesifik ditetapkan pada line2 dan m1, kemudian menggunakan karakter multi-level wildcard (#) di bagian akhir untuk menangkap seluruh sub-topik atau parameter sensor turunan di bawah mesin tersebut.

3. Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut:

Ekspresi topik subskripsi: pabrik/# Penjelasan: Karakter # yang ditempatkan langsung setelah akar domain pabrik/ akan mencakup dan menangkap seluruh pesan turunan apa pun secara keseluruhan tanpa terkecuali.

Latihan 3 (Implementasi Serialisasi Data Dinamis):

Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:

```
// Alokasi dokumen JSON statis dengan kapasitas yang disesuaikan
  StaticJsonDocument<256> doc;
  Doc[“node_id”] = clientId;
  Doc[“suhu”] = serialized(String(t, 2));       // Format desimal 2 angka di belakang koma
  Doc[“kelembapan”] = serialized(String(h, 2));

  // Penambahan parameter dinamis sesuai Latihan 3
  Doc[“uptime_detik”] = millis() / 1000;         // Waktu aktif sistem dalam detik
  Doc[“ip_lokal”] = WiFi.localIP().toString();   // Alamat IP lokal mikrokontroler
```

Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking):

Pada Praktikum 3, di dalam fungsi loop() terdapat pemanggilan rutin client.loop()

1. Konsekuensi teknis penyisipan delay(6000) di dalam fungsi loop() terhadap responsivitas penerimaan perintah callback:

- Penyisipan instruksi penundaan waktu yang lama seperti delay(6000) akan memblokir (blocking) seluruh eksekusi program di dalam fungsi loop(), sehingga mikrokontroler tidak dapat menjalankan fungsi client.loop() selama 6 detik tersebut.

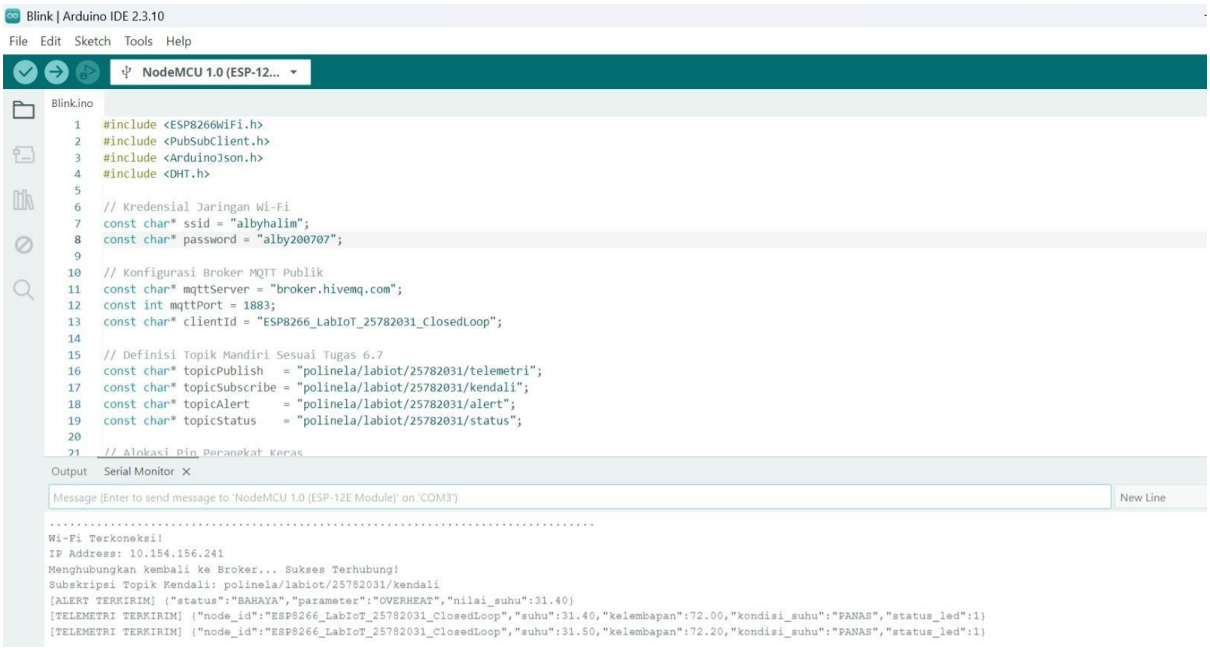
- Akibatnya, paket data atau perintah kontrol asinkron yang dikirimkan oleh broker melalui fungsi callback tidak dapat diproses secara seketika, menyebabkan latensi yang tinggi atau kegagalan respons pada LED indikator.
- Selain itu, jeda waktu yang melebihi batas toleransi ini akan memutuskan koneksi TCP secara paksa (timeout) karena paket keep-alive ping gagal dikirimkan tepat waktu ke MQTT broker.

2. Cara kerja fungsi reconnect() memulihkan koneksi saat router Wi-Fi atau broker MQTT sempat terputus sesaat:

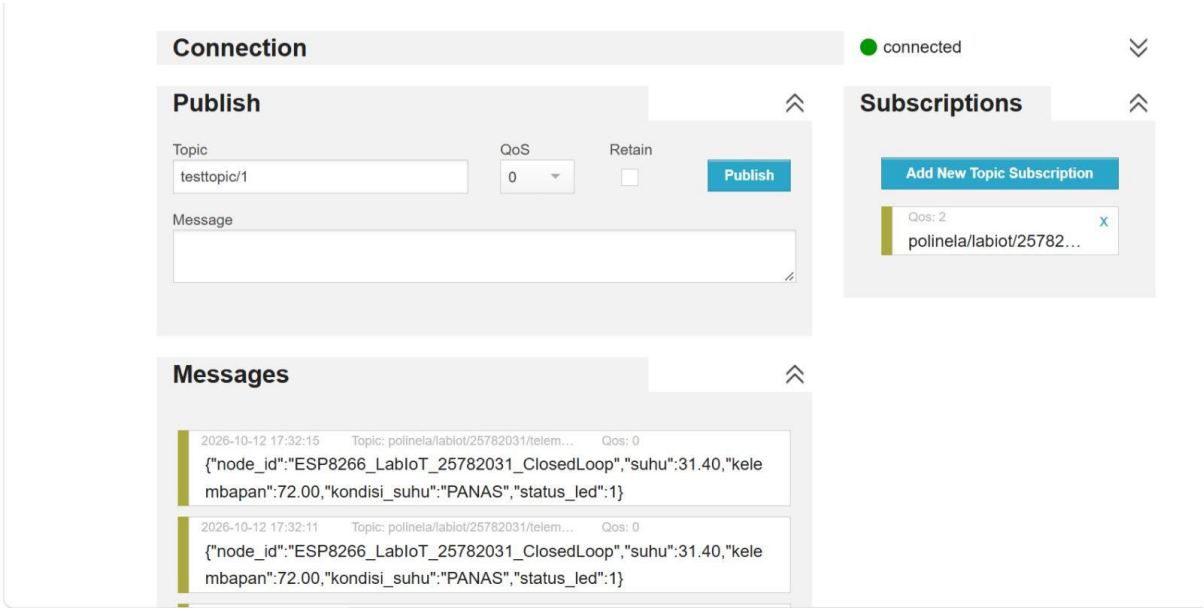
- Fungsi reconnect() beroperasi secara iteratif dengan memeriksa status koneksi menggunakan kondisional while (!client.connected()).
- Saat terdeteksi terputus, fungsi ini mencoba melakukan jabat tangan koneksi ulang ke broker menggunakan client.connect(clientId) (beserta parameter kredensial LWT jika dikonfigurasi).
- Jika koneksi berhasil dipulihkan (client.connect bernilai true), fungsi ini secara otomatis mendaftarkan ulang subskripsi topik (re-subscribing) agar simpul mikrokontroler dapat kembali mendengarkan dan menerima aliran pesan atau perintah kontrol tanpa intervensi manual.

6.7 Tugas

1. Klarifikasi status lingkungan pada telemetri rutin



Disini adalah monitoring suhu, perangkat dht mendeteksi bahwa lingkungan sedang panas dan mengeluarkan output parameter “overheat” karena suhu berada Pada >31.0C jika berada di <31.0C masuk kategori normal



User juga dapat memantau pada broker dan hasilnya akan terlihat Disini

2. Kanal peringatan Bahaya Mandiri

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...

Blink.ino

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// Kredensial Jaringan Wi-Fi
const char* ssid = "albyhalim";
const char* password = "alby200707";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782031_ClosedLoop";

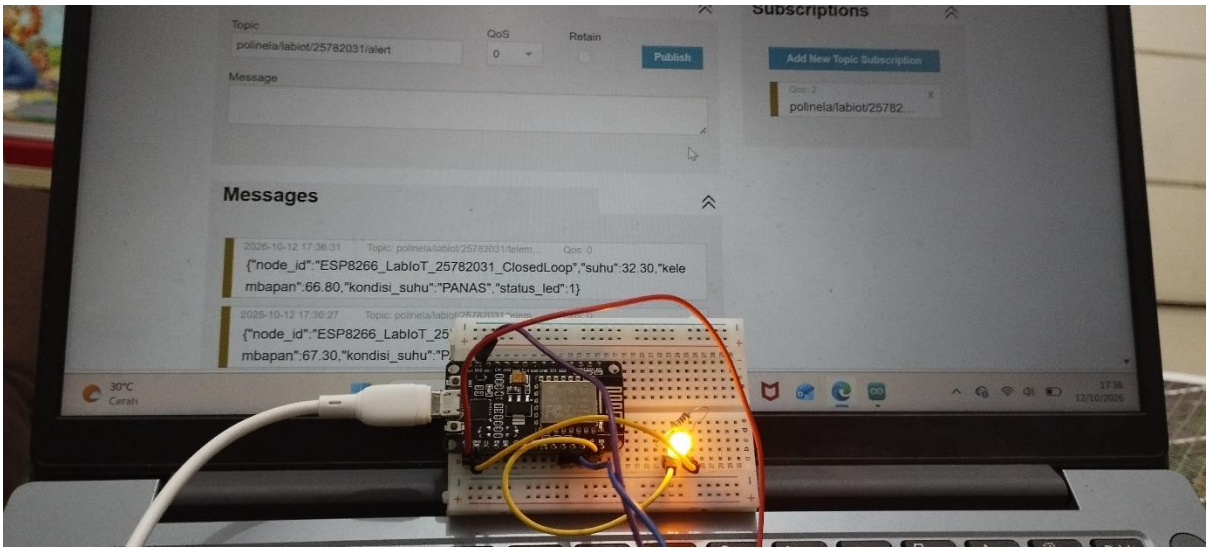
// Definisi Topik Mandiri Sesuai Tugas 6.7
const char* topicPublish   = "polinela/labiot/25782031/telemetri";
const char* topicSubscribe = "polinela/labiot/25782031/kendali";
const char* topicAlert     = "polinela/labiot/25782031/alert";
const char* topicStatus    = "polinela/labiot/25782031/status";

// Alokasi Pin Perangkat Keras
```

Output Serial Monitor

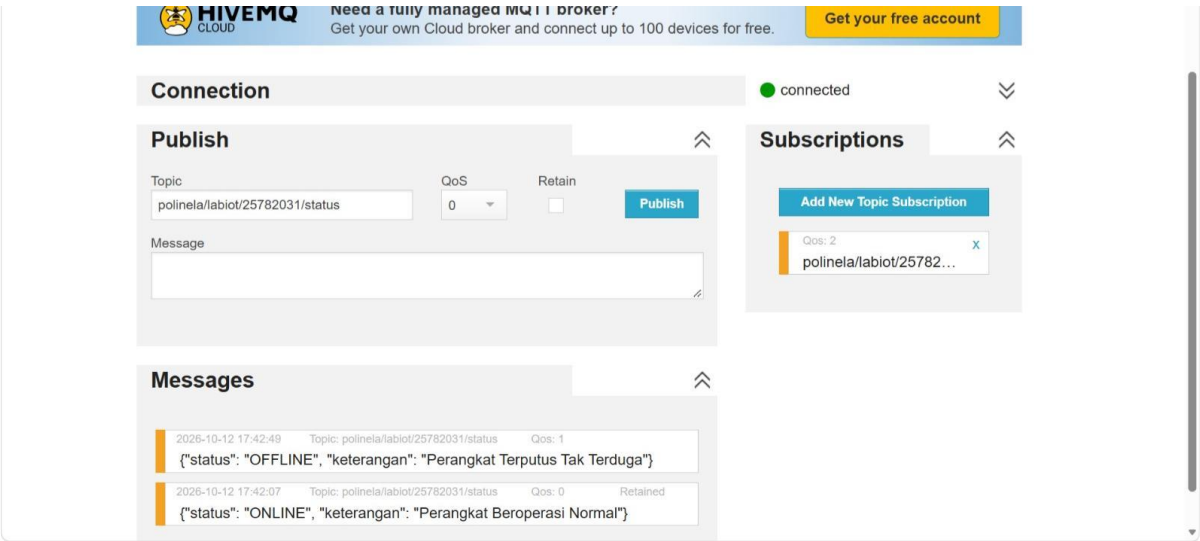
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

```
.....
Wi-Fi Terkoneksi!
IP Address: 10.154.156.241
Menghubungkan kembali ke Broker... Sukses Terhubung!
Subskripsi Topik Kendali: polinela/labiot/25782031/kendali
[ALERT TERKIRIM] {"status":"BAHAYA","parameter":"OVERHEAT","nilai_suhu":31.40}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782031_ClosedLoop","suhu":31.40,"kelembapan":72.00,"kondisi_suhu":"PANAS","status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782031_ClosedLoop","suhu":31.50,"kelembapan":72.20,"kondisi_suhu":"PANAS","status_led":1}
```

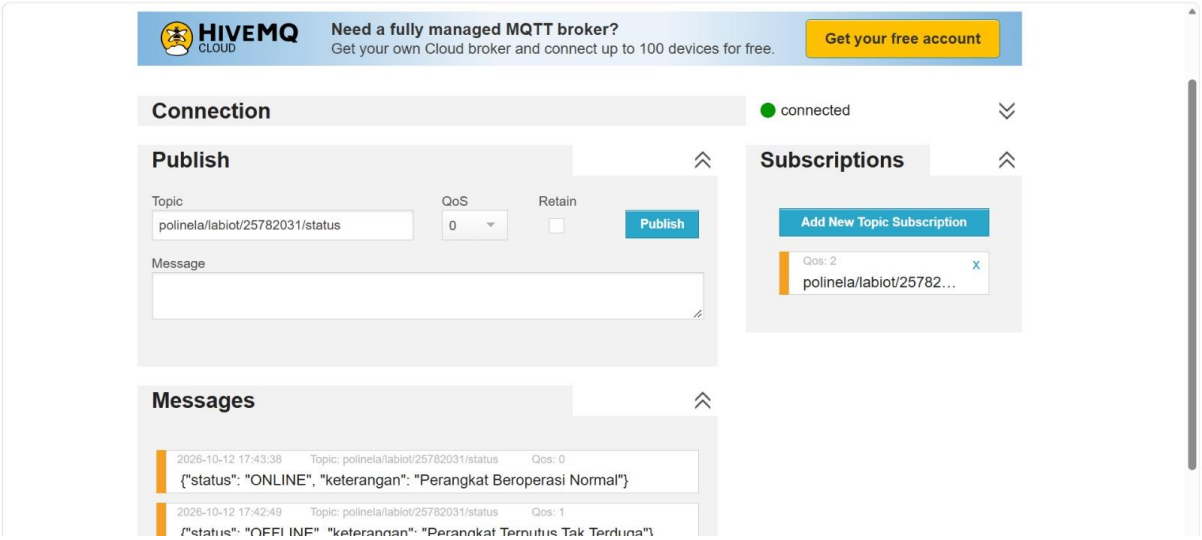


Pada saat dht22 mendeteksi suhu panas “overheat” yaitu berada pada 31.0C maka LED akan otomatis menyala dan menampilkan peringatan oeverheat dan juga panas dan bahaya

3. Mekanisme Last Will and Testament

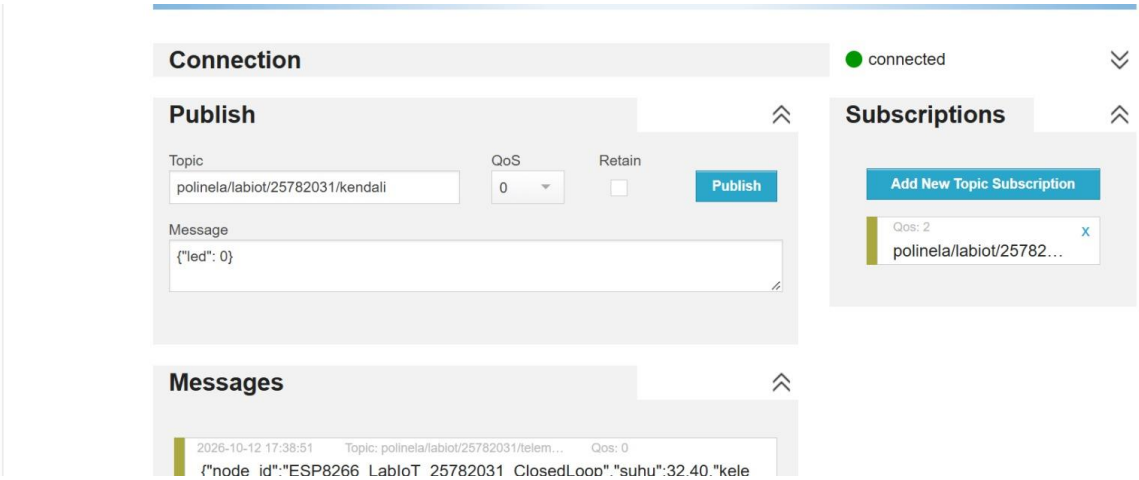


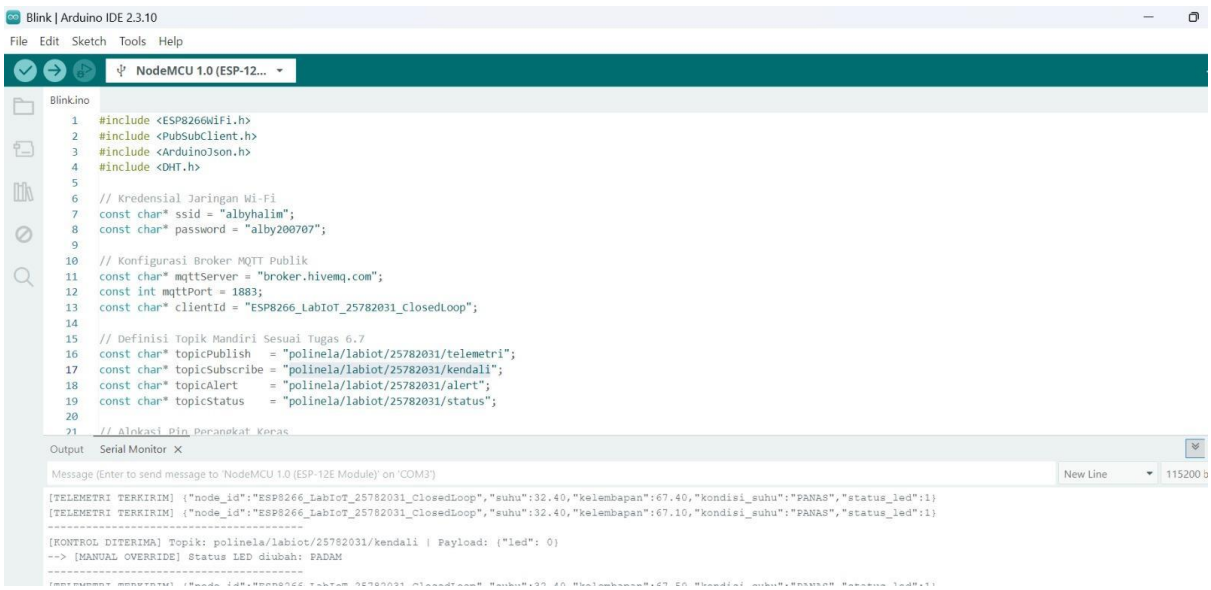
Masuk ke topik kampus/labiot/nim_anda/status pada Hivemq, lalu ekperimenkan pada bredboard dan cabut kabel data pada bredboard, nantinya pesan yang akan ditampilkan adalah status offline dan keterangannya “perangkat terputus tak terduga'



Lalu jika user memasang kabel data pada Bredboard kembali maka broker akan mencatat status perangkat online dan perangkat akan kembali beroperasi normal

4. Kendali Manual Override Saat Normal





User juga bisa melakukan Override manual ketika mengisi message pada broker dengan format JSON 0 maka LED akan padam